Assignment - II

Please ensure that your solutions are consolidated into a single zip file, with your roll number serving as the filename, and submitted prior to the specified due date as mentioned. Only one member from each group is designated to make the submission. Add instructions for running the solutions to each problem in separate files named `problem_X.txt`*. Alongside the solutions, a comprehensive report must be included, containing details such as program logic, pseudocode explanations, runtime analysis, relevant screenshots, and any other relevant information. During the evaluation, if necessary, a group shall be asked to arrange a demonstration of their submitted solutions with one of the TAs. All members of the group are expected to participate in this demonstration. Furthermore, during the session, each individual member may be assessed independently.*

MapReduce & Apache Hadoop

Problem 1. Co-occurring word matrix generation: Pairs & Stripes and local aggregation **14 Points**

a Develop a MapReduce program aimed at identifying the top 50 most frequently occurring words from the provided Wikipedia dump located on LMS (`Wikipedia-EN-20120601_ARTICLES.tar.gz`). Utilize the *pairs* approach for this task. Exclude non-stop words while computing the frequency of occurrence. Utilize the distributed caching option within Hadoop, demonstrated in `WordCount2.java`, to eliminate all stop-words in the mapper. (3 marks)

b Write another MapReduce program, employing the *pairs* approach, to construct the *co-occurring word matrix* based solely on the frequent words identified in the previous question. Consider a word distance of d when determining co-occurring pairs. Report the runtime for $d = \{1, 2, 3, 4\}$. (3 marks)

c Develop a *stripe* algorithm to create the co-occurring word matrix. Again, report the runtime for $d = \{1, 2, 3, 4\}$. (3 marks)

e Additionally, implement local aggregations separately at the Map-class level and the Map-function level, and compare their performance. Conduct this comparison for parts (b) and (c). Once again, the runtime must be reported for $d = \{1, 2, 3, 4\}$. (5 marks)

Note that these programs are expected to require a considerable amount of time to execute on your local system. It is advisable to plan your experiments in advance. Ensure that adequate screenshots are provided in the report to demonstrate execution time, among other relevant metrics. Mere submission of code and a few sample runtimes does not guarantee full marks.

Problem 2. Indexing Documents via Hadoop **12 Points**

a Implement a pair of a Map and a Reduce function which, for each distinct term that occurs in any of the text documents in `Wikipedia-EN-20120601_ARTICLES.tar.gz`, counts the number of distinct documents in which the term appears. We will call this value the Document Frequency (DF) of that term in the entire set of Wikipedia articles. Store the resulting DF values of all terms in a single TSV file with the following schema:

```
TERM<tab>DF
```

While generating the output in the above format, consider filtering out all terms that belong to the `stopwords.txt` file shared on LMS. (You may perform this filter operation in your map method.) Identify the top 100 terms with a high document frequency. Use those terms alone for the next sub problem.

Hint: Also here consider the Porter stemmer that is available from `opennlp-tools-1.9.3.jar` on LMS. After importing the library with `import opennlp.tools.stemmer.PorterStemmer;` and creating an instance of `PorterStemmer stemmer = new PorterStemmer();` use `stemmer.stem(token);` for stemming an input token. **6 Points**

b Implement another pair of a Map and a Reduce function which, for each document in `Wikipedia-EN-20120601_ARTICLES.tar.gz`, first counts the number of occurrences of each distinct term within the given document. We will call this value the Term Frequency (TF) of that term in the given document. Implement a stripes algorithm here as we are dealing with just 100 terms.

In a second step, your combined MapReduce function should multiply the TF value of each such term with the inverse of the logarithm of the normalized DF value calculated by the previous MapReduce function, i.e., $\text{SCORE} = \text{TF} \times \log(10000/\text{DF} + 1)$ for each combination of a term and a document. You may cache the former TSV file with the DF values by adding it via `Job.addCacheFile(<path-to-DF-file>)` in the driver function of your MapReduce program. The Map class should then load this file upon initialization into an appropriate main-memory data structure.

The result of this MapReduce function should be a single TSV file that has the same schema as the file we used in the previous exercise sheets: **6 Points**

```
ID<tab>TERM<tab>SCORE
```

Hint: You may use either the given document URL's or simple integer id's for the ID field of this TSV file (as long as they uniquely identify the original text article).